SNIA SDC: StorageAI 2026 - AiSIO: Orchestrating Storage I/O Across CPUs and Accelerators

Overview

A Samsung researcher presents AiSIO (Accelerator-integrated Storage I/O, also called ACIO), an open-source project for orchestrating NVMe storage I/O across CPUs and GPU accelerators. The talk covers three software challenges that bottleneck accelerator I/O, three supported I/O modes, the architecture of a new implementation replacing an earlier prototype, an overview of the open-source component ecosystem, and benchmark results from a 16-drive Dell server hosted at Samsung Memory Research Center.

Topics covered

- Three software challenges identified: (1) per-layer overhead across applications, libraries, language runtimes, user/kernel boundaries, and the OS storage stack; (2) unwanted data copies bouncing through host DRAM before reaching the accelerator; (3) the single physical function limit on storage devices, which restricts each device to one associated driver.
- Three I/O modes supported: CPU initiated (data to host memory), CPU initiated peer-to-peer (CPU drives transfer, data lands in accelerator memory), and device initiated peer-to-peer (accelerator self-initiates, data lands in its own memory).
- Design goal stated as interoperability over replacement: extend the existing software stack, preserve existing abstractions, and do so without trading off performance.
- All components are either already upstream in the Linux kernel or targeting upstream as open-source projects.
- Related work cited: Nvidia GDS, BAM (Big Accelerator Memory), Nvidia Scattera, AMD ROCk XIO, and libenvm; benchmarking

showed that Nvidia GDS with a single thread at small I/O sizes could not saturate even a single NVMe device.

- Initial prototype used SIOV (multiple physical functions) to run a kernel stack path and an ACIO prototype path simultaneously on one drive, with a file extend cache allowing the GPU to load LBAs directly.
- New architecture replaces libvm and custom kernel patches; main components are HOMIE (Host Orchestrated Multipath IO Daemon), the Extend Access Library (caching F_MAP_AP results), UDMA buff import, and the Ublock infrastructure for single-function device fallback.
- F_MAP_AP replaces raw XFS on-disk format decoding for file extent lookup; EBPF traces are planned for change notification.
- Open-source components: XNME (cross-platform I/O library with AiSIO primitives), XME Perf (benchmarking tool), UPC (MMIO doorbell module), Field Path (file-level benchmark), CHO (provisioning tool).
- A DMA buff infrastructure tap module is not yet upstream and is awaiting Linux kernel community feedback.
- Benchmark setup: 16 NVMe drives, unallocated mode, Dell server, Samsung Memory Research Center; SPDK BDEV Perf baseline: 62 million IOPS with 8 CPU cores.
- Application-layer tweaking via XME Perf raised throughput to 32 million IOPS with 3 hardware threads; driver-layer tweaking with UPCE tooling reached approximately 50 million IOPS on a single CPU with 2 hardware threads.
- CPU efficiency improved from 8 cores to 1.5 cores equivalent for the same 62 million IOPS workload.
- CPU initiated peer-to-peer and device initiated peer-to-peer both reach the same IOPS ceiling as the host-memory baseline; NVMe drive DMA cost is identical regardless of whether the DMA target is host memory or GPU memory.
- Peer-to-peer bandwidth: 3 NVMe drives saturate the PCIe Gen 5 GPU link at 4K I/O size; at 4K, a single CPU can saturate 8 GPUs in bandwidth.
- Q&A: files in the benchmark were pre-allocated and fixed size; the data-loader workload (image dataset including a TikTok dataset, 8 GB files) was inspired by the Nvidia DALI framework.

People, organizations, products

People: - Kristoff (audience member who objected to the F_MAP_AP approach during the talk)

Organizations: - Samsung - Nvidia - AMD - Dell

Products: - AiSIO (ACIO) - HOMIE (Host Orchestrated Multipath IO Daemon) - Nvidia GDS (GPU Direct Storage) - BAM (Big Accelerator Memory) - Nvidia Scattera - AMD ROCk XIO - SPDK (BDEV Perf, NVME Perf) - XNME - XME Perf - UPC - Field Path - CHO - Extend Access Library - Ublock - libenvm - UPCE - Nvidia DALI

Numbers and data points

- 3.8 million IOPS per NVMe drive in unallocated mode
- 62 million aggregate IOPS with 8 CPU cores, 16 NVMe drives (SPDK BDEV Perf baseline)
- 7.7 million IOPS per core at baseline (linear scaling)
- 6.3 million IOPS — BDEV Perf, 1 hardware thread
- 7.4 million IOPS — BDEV Perf, 2 hardware threads
- 16 million IOPS — BDEV Perf, 3 hardware threads on 2 CPU cores
- 32 million IOPS — XME Perf, 3 hardware threads (application-layer tweaking)
- approximately 40 million IOPS — driver-layer tweaking, 1 hardware thread
- approximately 50 million IOPS — driver-layer tweaking, 1 CPU / 2 hardware threads
- 1.5 cores equivalent after optimization to sustain 62 million IOPS
- 3 NVMe drives saturate PCIe Gen 5 GPU link at 4K I/O
- 1 CPU saturates 8 GPUs in bandwidth at 4K I/O
- 8 GB — size of large pre-allocated files in data-loader benchmark

Notable quotes

> basically we want to extend the existing software stack and we want to do so in ways where we’re not trading off the utilities of having existing abstractions and we don’t want to trade off performance either. — unknown

> that’s the vision. We want to be able to have accelerators as uh uh first class citizens in the um in the storage stack. — unknown

> a single thread small IO we cannot saturate even a single NVME device using this. — unknown

> we hit the 62 million IOPS here with eight CPU cores. So that's 16 devices. Uh that gets saturated by using eight CPU cores. — unknown

> the NVME drive doesn't really care that the DMA address is on your host memory or on your GPU. The cost of processing that IO is the same. — unknown

> is a proper way to do it and I told for two years to multiple members in your team how to do it and it's really upsetting that you can keep publishing kind of dangerous — Kristoff

> debugging tool. It has different content for different file systems. It's a massive risk for uh cost and corruption due to concurrent activity. That's why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients — Kristoff